

Facundo Gael Piñeiro González

A01666626

3 de Junio de 2026

Evidencia/Reflexión: STC0204 Desarrollo de componentes de software

Componentes fundamentales para el desarrollo de un software

1. FRONT-END:

Es la vista principal de la página, aquí se juntan los estilos y funciones ya definidas en el CSS y JS.

HTML - Archivo de ejecución de una página web, esta dividido principalmente por head y body. Head es donde se ponen formatos y referencias de estilo, por ejemplo, formato de escritura tipo UTF-8 para traducir caracteres a código binario que la computadora pueda entender, o referencias a frameworks como Bootstrap o al archivo de estilos style.css.

CSS - Archivo que sirve para ejecutar variables con formato y color, dándole estilos a los elementos que se utilizarán en HTML o EJS.

EJS - Similar a html pero con la característica de poder incluir lógica del back-end directamente en la vista, permitiendo renderizar datos dinámicos desde el servidor, como lista de usuarios, mensajes de error o contenido personalizado según el rol de un usuario.

2. BACK-END:

Son todas las funcionalidades que ocurren detras de camaras. Se pueden crear funciones que interactuan con el front-end, como la redirección hacia otras páginas, el envío de datos a la base de datos, la asignación de roles a usuarios, etc.

JavaScript - Lenguaje de programación que se utiliza tanto en el front-end (EJS) como en el back-end (JS). Define la lógica del negocio, maneja rutas y gestiona la comunicación con la base de datos. Este es el código principal utilizado para nuestro proyecto.

[NODE.JS:](#)

Base de cualquier back-end. Es un entorno de ejecución de código abierto que permite ejecutar JS en el servidor. Al igual que JS, gestiona rutas, procesa información y comunica a la base de datos. Cuenta con librerías, cada una con funciones adicionales.

PNPM:

Herramienta que administra las dependencias del back-End, como las librerías y paquetes externos. Es más eficiente que NPM ya que evita duplicar paquetes entre proyectos. También cuenta con librerías al igual que NPM.

Herramientas / comandos utilizadas en clase y en el proyecto:

Herramienta	Comando Node.js	Comando pnpm	Desc./ Utilización
Instalación	npm i / npm install	npm i pnpm -g	instala las dependencias del proyecto
versión	npm --version	pnpm --version	verifica versión
Express	npm i express ejs	pnpm add express	framework principal del servidor web
body-parser	npm i body-parser	-	interpreta peticiones HTTP
cookie-parser	npm i cookie-parser	-	crear y gestionar cookies
express-session	npm i express-session	-	maneja sesione de usuario en el servidor
csrf-csrf	npm i csrf-csrf	-	protección vs CSRF
helmet	npm i helmet	-	cabeceras de seguridad
dotenv	npm i dotenv	-	carga el entorno .env
pg	npm i pg	-	conecta a PostgreSQL
bcrypt	npm i bcrypt	-	ayuda a encriptar contraseñas
multer	-	pnpm add multer	ayuda a gestionar la subida de archivos al servidor

3. BASE DE DATOS:

Donde se guarda la información de las peticiones realizadas dentro del servidor. Por ejemplo, al registrar un usuario se genera una ID y se almacenan datos como nombre, correo y contraseña. Al iniciar sesión, el sistema consulta la base de datos para comprobar si los datos ingresados corresponden a un registro existente.

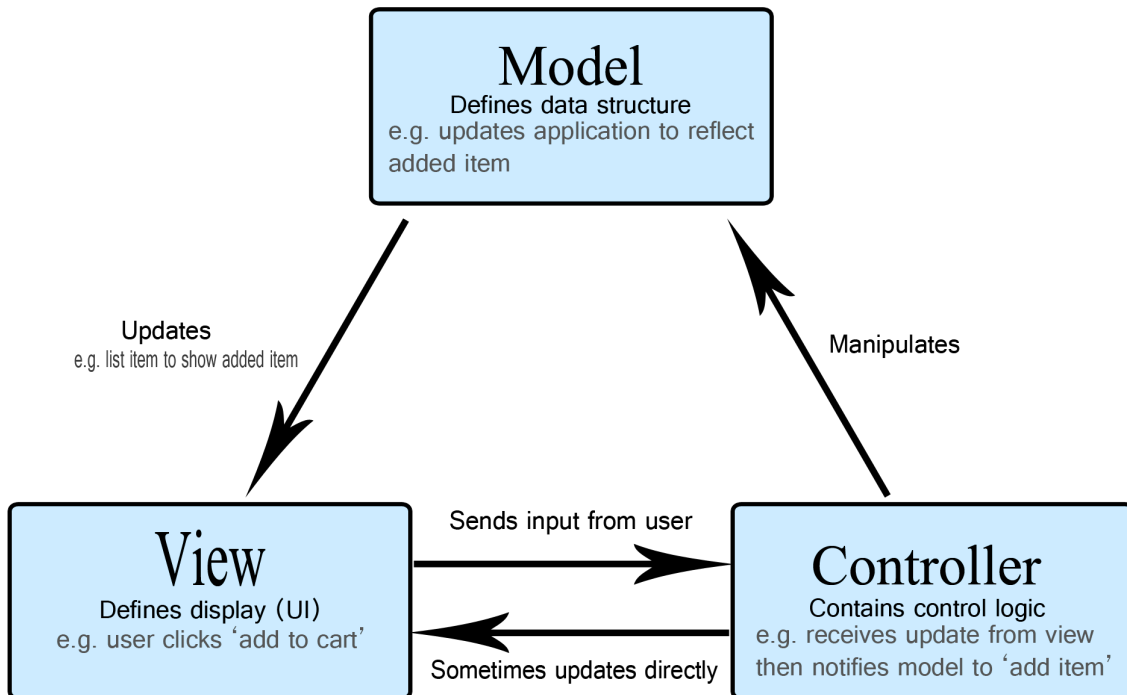
Una de las características entre aplicaciones de SQL es que cuentan con un lenguaje procedimental o extensiones procedimentales, las cuales sirven para combinar sentencias SQL estandar con estructuras de control de flujo.

La base de datos que estamos utilizando en el proyecto es PostgreSQL, tiene como lenguaje procedimental nativo PL/pgsql

4. ARQUITECTURA MVC:

La arquitectura MVC (Models-Views-Controllers) es una arquitectura de desarrollo de software que divide el proyecto en 3 capas:

- Model - Es la capa de información y comunicación. Se comunica directamente con la base de datos.
- View - Es la capa de presentación, corresponde a los archivos EJS que el usuario visualiza.
- Controller - Capa intermediaria entre Model y View, recibe las peticiones de los usuarios, las procesa y devuelve una respuesta.



5. CONVENCIONES DE CÓDIGO:

Las convenciones de código son básicamente reglas para tener un código legible y consistente. Dentro del equipo definimos estas convenciones para el proyecto:

- Utilización de arquitectura MVC para una mejor organización del proyecto.
- Uso de nombres descriptivos en variables, funciones y archivos
- camelCase para variables y funciones tipo `productName` y `userId`.
- Asignación de responsabilidad y roles en las rutas, modelos, controles y vistas.
- Creación de archivo `.env` para almacenar información sensible.
- Formato de código limpio.

Evidencias:

En las etapas tempranas del proyecto, trabajé más que nada en la traducción de los diagramas a código SQL, posteriormente creando las tablas y queries que conforman el sistema RBAC. En las últimas semanas contribuí más que nada a que el back-end pueda enviar formalmente peticiones a la base de datos, configurando las

querry que estan dentro de unos archivos de controller como usuarios.controller o modificando los permisos que unos tienen al entrar a páginas en la carpeta de rutas.

- Ejemplo de contribuciones:

Verificación de roles en rutas

routes/[alertas.routes.js](#)

```
1  const express = require("express");
2  const router = express.Router();
3  const alertasController = require("../controllers/alertas.controller");
4  const isAuthenticated = require("../config/is-auth");
5  const verificarRol = require("../config/verificarRol");
6
7  const ROLES_ADMIN = ["Administrador", "Oficial_Cumplimiento"];
8
9  router.get("/alertas", isAuthenticated, verificarRol(ROLES_ADMIN), alertasController.renderAlertas);
10
11 module.exports = router;
```

routes nos ayuda a redirigir usuarios a otros sitios dentro del servidor, el cambio esta más que nada en la modificación para hacer que un usuario pueda conectarse a una página dependiendo si tiene un rol o no.

¿Como?

ayudandonos de las configuraciones de rbac que estan situadas en la carpeta de config, podemos llamarlas y utilizarlas para simplemente decir que con los roles creados, asignamos que si son verificados, que puedan ir a la página.

Este formato esta implementado en casi todas las rutas del proyecto, chisme tiene varias verificaciones, una por si eres cliente, otra por si eres empleado y otra por si eres admin.

Implementación de variación en /testing

public/cliente.ejs

```

1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/css/bootstrap.min.css" rel="stylesheet">
7   <link rel="stylesheet" href="/styles.css">
8   <title>Portal del Cliente</title>
9 </head>
10 <body class="standalone">
11   <X- include("partials/topbar") %>
12 <main>
13   <section class="panel full">
14     <h1>Envío de Documentos</h1>
15
16     <form action="/upload_file_private" method="POST" enctype="multipart/form-data">
17       <label>Nombre completo:</label>
18       <input type="text" name="nombre">
19
20       <label>RFC:</label>
21       <input type="text" name="rfc" maxlength="13">
22
23       <label>CURP:</label>
24       <input type="text" name="curp" maxlength="18">
25
26       <label for="file_input">Subir Documento</label>
27       <input id="file_input" type="file" name="file" accept=".pdf,.jpg,.jpeg,.png">
28
29       <button type="submit">Enviar</button>
30     </form>
31   </section>
32 </main>
33 </body>
34 </html>

```

controllers/chisme.controller.js

```

exports.renderChisme = (req, res) => {
  const roles = req.session.usuario?.roles || [];

  if (roles.includes("Cliente")) {
    return res.render("cliente", { pageTitle: "Portal del Cliente" });
  }

  res.render("chisme", {
    pageTitle: "Chisme - Beta 1"
  });
};

```

routes/chisme.routes.js

```

1 const express = require("express");
2 const router = express.Router();
3 const controller = require("../controllers/chisme.controller");
4 const isAuth = require("../config/is-auth");
5 const verificarRol = require("../config/verificarRol");
6
7 const ROLES_ADMIN = ["Administrador", "Oficial_Cumplimiento"];
8 const ROLES_TODOS = ["Administrador", "Oficial_Cumplimiento", "Empleado", "Cliente"];
9
10 router.get("/testing", isAuth, verificarRol(ROLES_TODOS), controller.renderChisme);
11 router.post("/upload_file", isAuth, verificarRol(ROLES_TODOS), controller.upload_file);
12 router.post("/upload_file_private", isAuth, verificarRol(ROLES_ADMIN), controller.upload_file_private);
13 router.get("/get_private_file/:file", isAuth, verificarRol(ROLES_ADMIN), controller.get_private_file);
14
15 module.exports = router;

```

/testing comparte 2 views,

Cliente (cliente.ejs) - permite a los clientes ingresar SOLAMENTE a este sitio para subir archivos como RFC, CURP, etc.

Empleado (chisme.ejs) - permite a los empleados ingresar con su correo para SOLO llenar el chismografo.

¿Cómo?

En [chisme.controller.js](#) se declara el export renderChisme, el cuál valida si el usuario autenticado es cliente, si lo es, renderiza cliente.ejs, de lo contrario, renderiza chisme.ejs.

Este export es posteriormente llamado en [chisme.routes.js](#) mediante `router.get("/testing", ..., controller.renderChisme)` para finalizar la verificación del rol antes de renderizar la vista correspondiente.

Modificaciones pendientes:

Lo que nos faltaria implementar en este aspecto es quitar vistas que el empleado tiene pero que no puede acceder debido a los chequeos en ruta, por lo que modificar la sidebar en su vista.

Reflexión:

Entender los componentes que conforman el desarrollo de un software me ayuda a entender una visión más clara de como iniciar un proyecto, uno como desarrollador, tiene que tratar de cumplirlos lo más que pueda. Sabiendo de antemano que dependencias, lenguajes y herramientas serán necesarias según el tipo de sistema a desarrollar.

Además, la organización mediante una arquitectura de un código como MVC es sumamente importante, ya que permite que todo el equipo trabaje sobre una estructura común y fácil de entender. Esto facilita el comunicar avances, modificar código de compañeros y apoyarse mutuamente cuando algo falla, sin perder tiempo tratando de entender como esta organizado el proyecto.